	**Carátula para entrega de prácticas**

Laboratorio de computación salas A y B

Profesor:	René Adrián Dávila Pérez	
Asignatura:	Programación Orientada a Objetos	
Grupo:	7	
No de Práctica(s):	11	
Integrante(s):	120002850	323064039
	323071378	323214636
	426059576	426058744
Semestre:	2027-1	
Fecha de entrega:	11-Noviembre-2026	
Obervaciones:		

CALIFICACIÓN:

_____

Índice

1. Introducción **3**
1.1. Planteamiento del problema 3
1.2. Motivación 3
1.3. Objetivos 3

2. Marco Teórico **4**
2.1. Cuadros de texto en interfaces gráficas: Swing versus JavaFX 4
2.2. Manejo de archivos y flujos de entrada/salida en Java NIO.2 5
2.3. Separación arquitectónica entre lógica de dominio e interfaz visual . . 5

3. Desarrollo **6**
3.1. Investigación comparativa: Espacio de cuadro de texto en Swing y JavaFX 6
3.2. Evidencias de capturas de investigación empírica 7
3.3. Arquitectura del software y especificación de clases (Ejercicio 2) . . . 12
3.3.1. Clase ContadorPalabras (Lógica de Dominio y Modelo) 12
3.3.2. Clase VentanaContador (Interfaz Gráfica y Controlador) . . . 13
3.4. Relaciones estructurales y diagramas UML 14
3.5. Dinámica algorítmica y diagramas de comportamiento 17

4. Resultados **20**
4.1. Caso 1: Representativo (Texto estructurado en español) 20
4.2. Caso 2: Frontera (Espaciados múltiples, tabuladores y caracteres especiales) 23
4.3. Verificación del estado inicial de la interfaz gráfica de usuario 25
4.4. Verificación de empaquetado en archivo JAR autónomo 26

5. Conclusiones **27**

1. Introducción

1.1. Planteamiento del problema

En el desarrollo contemporáneo de aplicaciones de escritorio sobre la plataforma Java, la interacción gráfica demanda controles visuales intuitivos integrados armónicamente con flujos de entrada y salida hacia el almacenamiento secundario. La construcción de un sistema para inspeccionar documentos de texto plano plantea el problema de aislar estrictamente la capa de presentación de la lógica algorítmica de análisis textual. El empleo de componentes multilínea en bibliotecas como Swing sin contenedores especializados propicia la pérdida de visibilidad ante desbordamientos de contenido, mientras que la lectura de archivos sin validaciones defensivas de extensión, existencia y codificación estándar ocasiona excepciones no controladas o cómputos erróneos en la cuantificación léxica. Por consiguiente, se requiere formular una solución modular, tipada y resiliente que resuelva con rigor la carga y el procesamiento de archivos de texto.

1.2. Motivación

El paradigma de programación orientada a objetos brinda principios esenciales de encapsulamiento y delegación para gobernar la complejidad del diseño visual y de la persistencia de datos. Contrastar técnicamente marcos consolidados como Swing frente a alternativas modernas como JavaFX, junto con el dominio de operaciones atómicas sobre el sistema de archivos mediante la API Java NIO.2, consolida competencias fundamentales en el estudiante de ingeniería de software. La implementación de un contador de palabras interactivo permite ejercitar la separación Modelo-Vista, el confinamiento seguro de eventos en hilos especializados y la normalización léxica mediante expresiones regulares, garantizando que el software permanezca íntegro y determinista ante entradas arbitrarias del usuario o pruebas automatizadas por terminal.

1.3. Objetivos

El objetivo principal de esta práctica consiste en diseñar, implementar, verificar y documentar una aplicación con interfaz gráfica en Java basada en Swing, acompañada de un análisis comparativo frente a JavaFX, para la lectura de archivos con extensión `.txt` y la cuantificación precisa de las palabras contenidas. De manera específica, se busca materializar la separación arquitectónica entre la clase de dominio (`ContadorPalabras`) y la interfaz gráfica (`VentanaContador`); verificar el comportamiento del sistema mediante casos de prueba representativos y de frontera; comprobar su empaquetado autónomo en un archivo JAR ejecutable; y documentar la investigación conceptual asistida por herramientas de inteligencia artificial generativa.

2. Marco Teórico

2.1. Cuadros de texto en interfaces gráficas: Swing versus JavaFX

La plataforma Java provee dos bibliotecas primordiales para la construcción de interfaces de usuario de escritorio: la biblioteca histórica Swing, ubicada en el paquete `javax.swing`, y la plataforma moderna JavaFX, estructurada bajo el paquete `javafx.scene.control` [1, 2]. La administración y visualización de cadenas de texto plano manifiesta discrepancias arquitectónicas determinantes entre ambos marcos de desarrollo.

En el ecosistema Swing, la gestión textual se bifurca entre `JTextField` para una sola línea y `JTextArea` para secuencias multilínea. Una particularidad nodal de `JTextArea` reside en que actúa como un componente ligero enfocado únicamente en manipular el modelo de documento (`Document`), careciendo intrínsecamente de barras de desplazamiento o mecanismos de visualización virtualizada [2]. Si el volumen textual excede los límites espaciales del componente en pantalla, los caracteres sobrantes quedan ocultos. Para proveer capacidades de navegación, la arquitectura de Swing exige envolver explícitamente a `JTextArea` en un contenedor decorador: la clase `JScrollPane`. Este último gestiona un área de visualización (*viewport*) y proyecta condicionalmente barras verticales y horizontales según políticas como `VERTICAL_SCROLLBAR_AS_NEEDED`.

Asimismo, Swing se basa en primitivas gráficas ligeras dibujadas por software en la CPU mediante llamadas 2D sobre AWT, sujetándose al principio de confinamiento de hilo en el Hilo de Despacho de Eventos (*Event Dispatch Thread*, EDT). Toda modificación visual debe canalizarse asíncronamente mediante la instrucción `SwingUtilities.invokeLater` para impedir anomalías de sincronización [2].

Por el contrario, JavaFX introduce controles autónomos dentro de la jerarquía `TextInputControl`, tales como `TextField` y `TextArea`. El control `TextArea` de JavaFX incorpora de manera nativa e integrada en su propio contenedor visual (*skin*) barras de desplazamiento y ajuste automático de línea (*line wrap*), prescindiendo por completo de envoltorios externos como `ScrollPane` para el uso general [3].

A nivel arquitectónico, JavaFX estructura sus interfaces sobre un Grafo de Escena (*Scene Graph*) procesado con aceleración por hardware en la GPU mediante el motor Prism (DirectX, OpenGL o Metal) [3]. Asimismo, JavaFX incorpora el enlace reactivo de propiedades (*Property Binding*), donde controles como `TextArea` exponen propiedades observables como `textProperty()`, lo que permite sincronizar modelos de datos y vistas de forma declarativa sin requerir escuchadores manuales de bajo nivel (`DocumentListener`). La apariencia se desacopla mediante hojas de estilo JavaFX CSS y definiciones FXML, orquestando las mutaciones de la escena en el *JavaFX Application Thread* mediante la instrucción `Platform.runLater`.

2.2. Manejo de archivos y flujos de entrada/salida en Java NIO.2

El acceso a los recursos de almacenamiento secundario en Java transitó desde las primitivas clásicas de flujo basadas en `java.io.File` y `BufferedReader` hacia la moderna API de entrada y salida sin bloqueo NIO.2, incorporada en el paquete `java.nio.file` [4].

La API NIO.2 estructura el acceso al sistema de archivos mediante la interfaz `Path` y la clase de utilidades `Files`. En particular, la operación atómica provista por `Files.readString` asegura la apertura, transferencia completa de bytes y cierre infalible del canal de lectura, previniendo fugas de descriptores de archivos incluso ante excepciones de entrada y salida (`IOException`) [4].

Asimismo, la especificación explícita de la codificación resulta indispensable para evitar inconsistencias entre plataformas. El empleo mandatorio de la constante `StandardCharsets.UTF_8` asegura la lectura homogénea de signos ortográficos, acentos diacríticos y caracteres Unicode, garantizando que el análisis léxico opere sobre representaciones deterministas [1].

2.3. Separación arquitectónica entre lógica de dominio e interfaz visual

El diseño orientado a objetos postula el aislamiento estricto de las reglas de negocio respecto a los mecanismos de presentación gráfica [5]. Bajo el patrón arquitectónico Modelo-Vista, el modelo de datos encapsula la validación de archivos, la lectura de bytes y los algoritmos de descomposición textual sin incluir dependencias hacia componentes gráficos como botones, etiquetas o ventanas.

Esta disciplina confiere portabilidad absoluta al modelo, permitiendo reutilizar la clase de análisis en entornos de terminal, pruebas unitarias automatizadas o microservicios. A su vez, delimita con claridad la gestión de fallos: el modelo señala contingencias mediante excepciones tipadas (`IllegalArgumentException`, `FileNotFoundException`, `IOException`), mientras que la vista se restringe a capturarlas y ofrecer retroalimentación comprensible al usuario sin corromper el estado del sistema.

3. Desarrollo

3.1. Investigación comparativa: Espacio de cuadro de texto en Swing y JavaFX

En atención al Ejercicio 1, se definió una consulta estandarizada preservada en el repositorio bajo `Capturas/query.txt`. Dicha consulta demanda una extensión acotada entre 90 y 120 palabras, omitiendo introducciones y despedidas, orientada a contrastar la gestión de cuadros de texto, concurrencia, enlace reactivo y estilización entre Swing y JavaFX.

A partir del análisis técnico efectuado, se sintetizan en la tabla 1 las divergencias fundamentales entre ambos paradigmas gráficos de la plataforma Java.

Criterio Técnico	Swing (`javax.swing`)	JavaFX (`javafx.scene.control`)
Componente de texto monolínea	`JTextField` con modelo `Document`.	`TextField` con texto guía y formateadores.
Componente multilínea	`JTextArea` sin barras de desplazamiento propias.	`TextArea` con desplazamiento nativo e integrado.
Soporte de desplazamiento	Exige envoltura manual obligatoria en `JScrollPane`.	Integrado de fábrica dentro del propio skin visual.
Enlace de datos y reactividad	Oyentes clásicos (`DocumentListener`, `CaretListener`).	Enlace reactivo automático mediante *Property Binding*.
Arquitectura de presentación	Jerarquía de componentes ligeros sobre AWT.	Grafo de Escena (*Scene Graph*) de nodos transformables.
Motor de renderizado	Procesamiento por software en CPU vía `Graphics2D`.	Aceleración por hardware en GPU vía motor Prism.
Estilización y diseño	*Look and Feel* (`UIManager`) y código imperativo.	Hojas de estilo JavaFX CSS (`-fx-*`) y marcado FXML.
Hilo principal de eventos	*Event Dispatch Thread* (`SwingUtilities.invokeLater`).	*JavaFX App. Thread* (`Platform.runLater`).

Tabla 1: Matriz comparativa sistemática entre los entornos gráficos Swing y JavaFX.

3.2. Evidencias de capturas de investigación empírica

A continuación se presentan las seis evidencias individuales obtenidas por los integrantes del equipo al ejecutar la consulta comparativa estandarizada en herramientas de inteligencia artificial, contrastando la gestión de cuadros de texto en Swing y JavaFX.

En la figura 3.1 se muestra la consulta realizada por Alan; en la figura 3.2, la de Zoé; y en la figura 3.3, la de Luis. Por su parte, en las figuras 3.4, 3.5 y 3.6 se integran las respuestas obtenidas por Joseph, Hanniel y Alex, respectivamente.

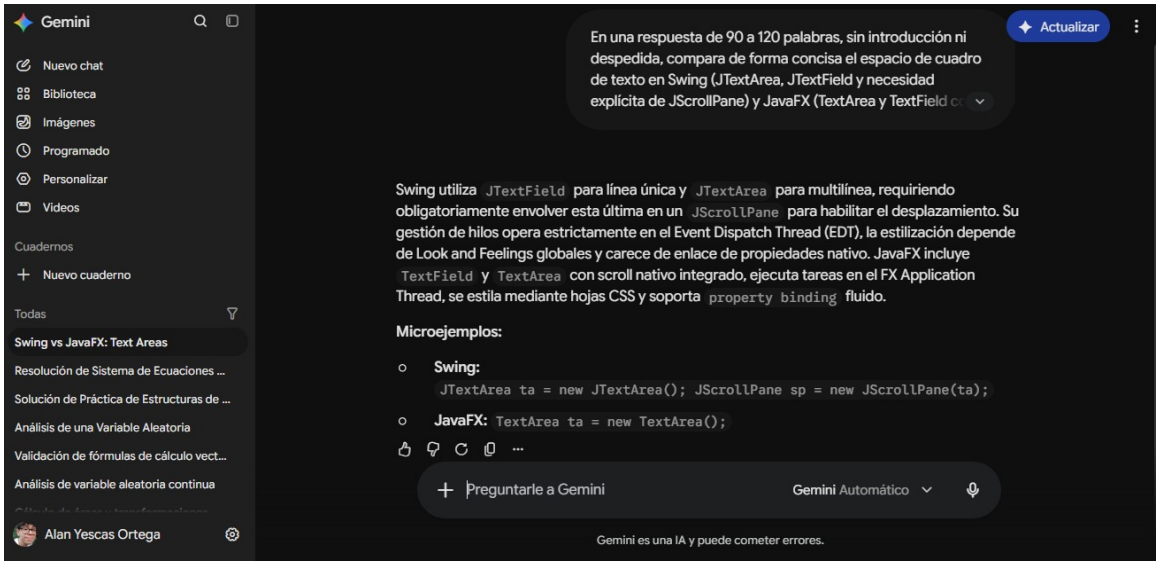


Figura 3.1: Consulta de investigación en IA realizada por Alan.

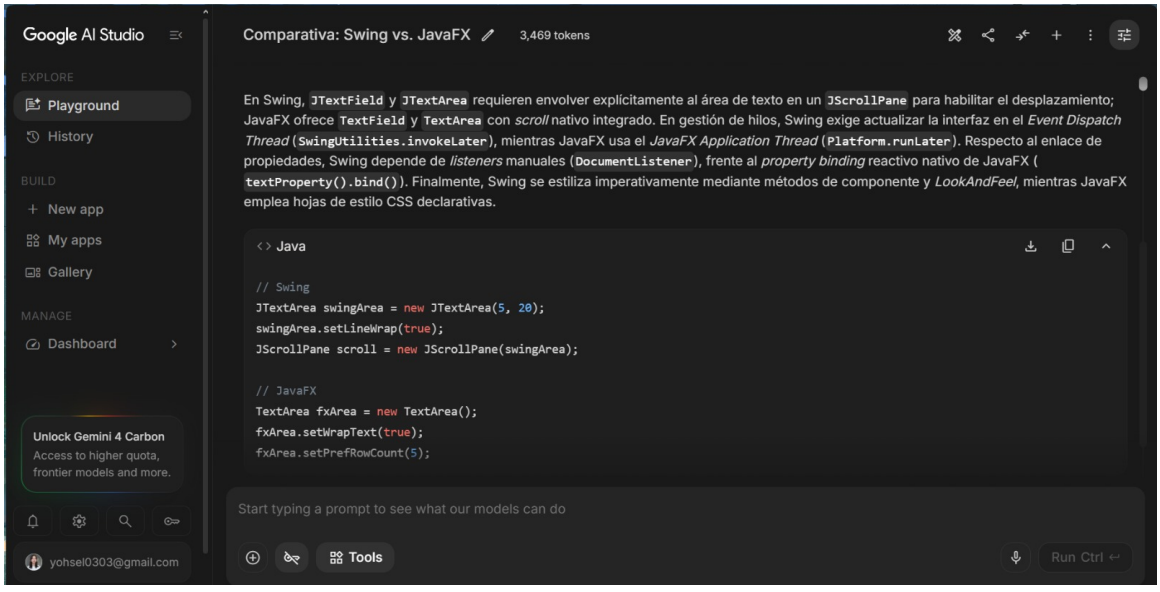


Figura 3.2: Consulta de investigación en IA realizada por Zoé.

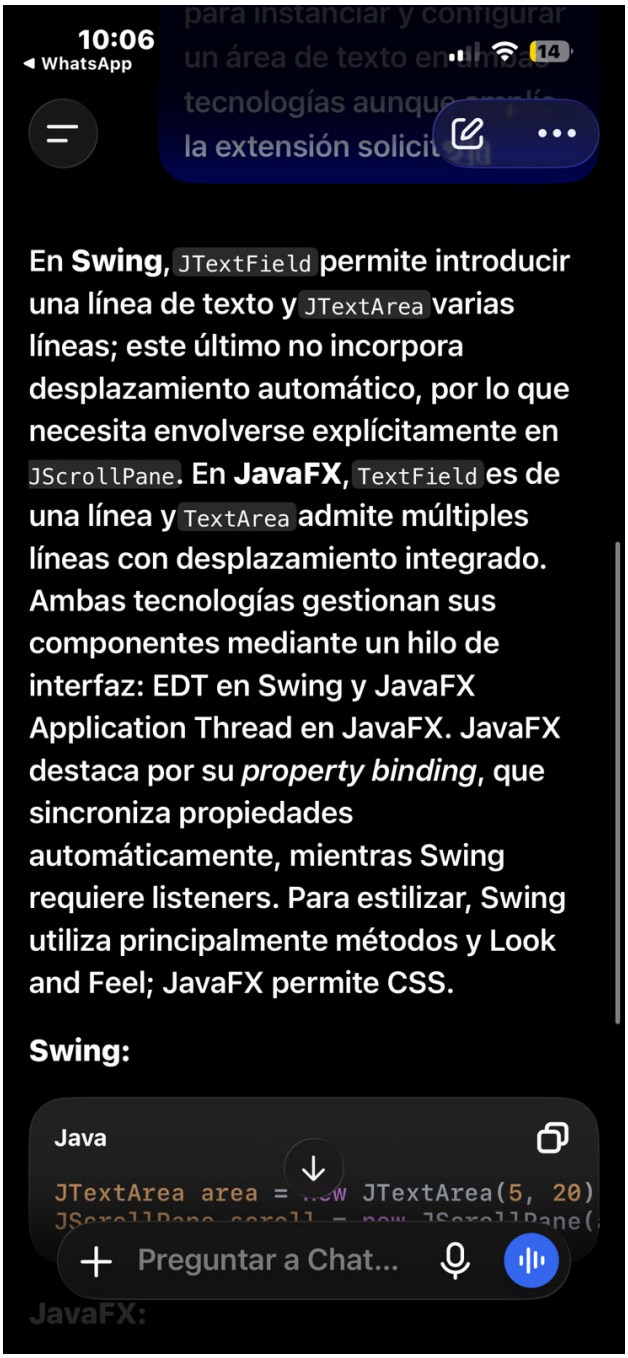


Figura 3.3: Consulta de investigación en IA realizada por Luis.

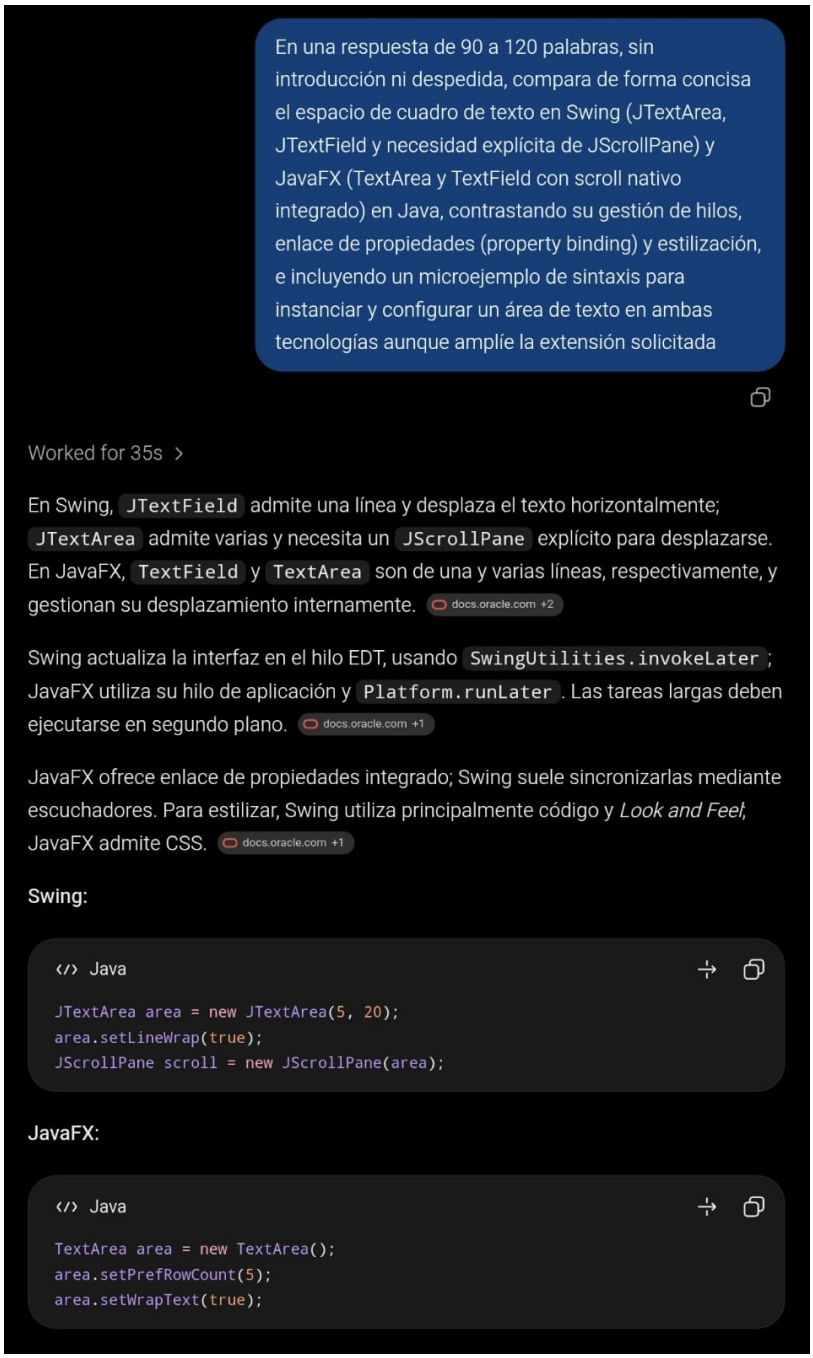


Figura 3.4: Consulta de investigación en IA realizada por Joseph.

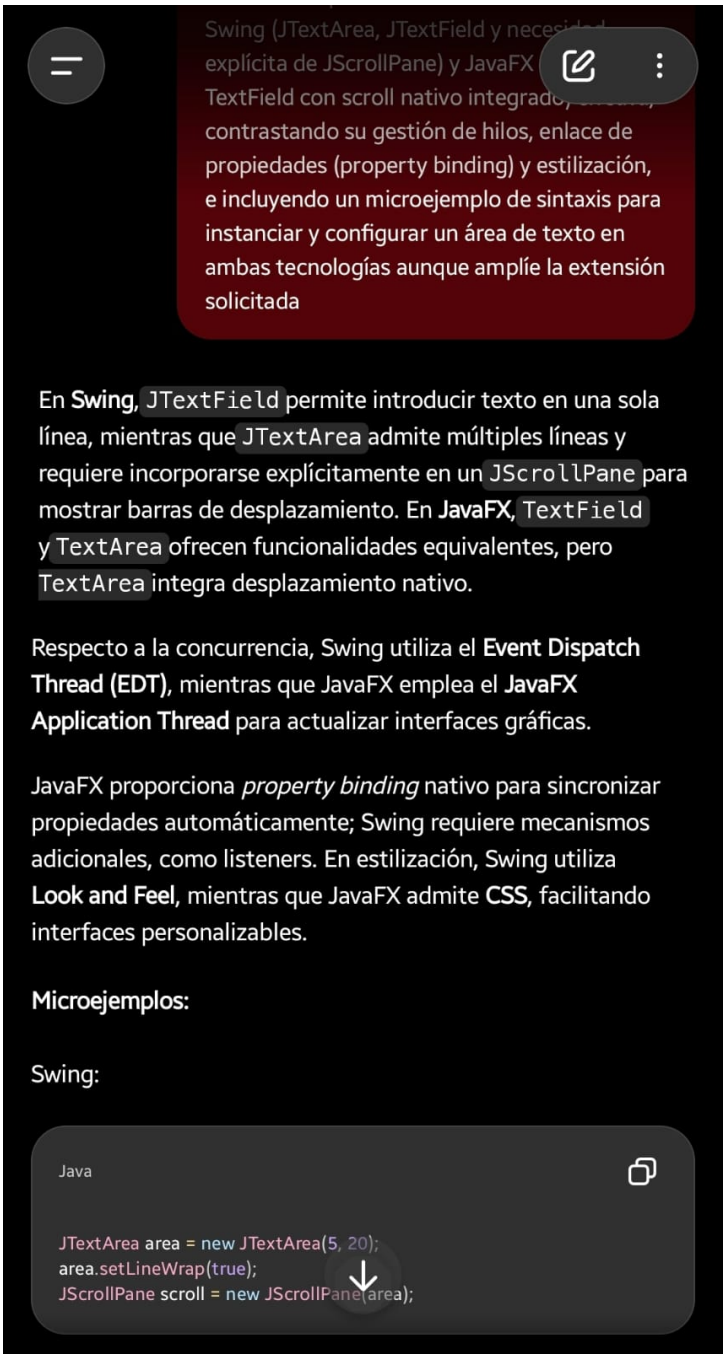


Figura 3.5: Consulta de investigación en IA realizada por Hanniel.

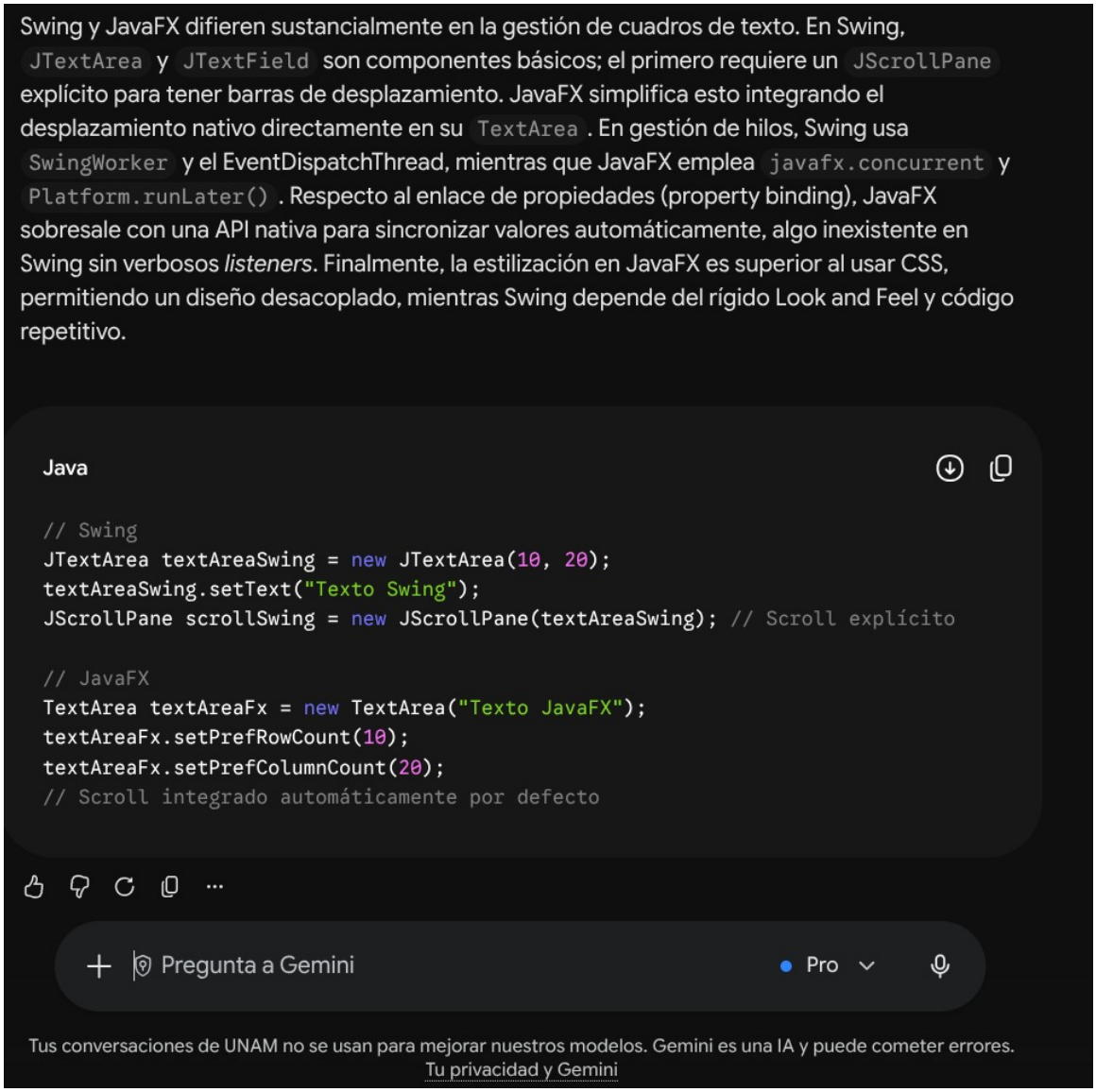


Figura 3.6: Consulta de investigación en IA realizada por Alex.

3.3. Arquitectura del software y especificación de clases (Ejercicio 2)

Para resolver la lectura interactiva de archivos de texto plano y el cómputo riguroso de palabras conforme al Ejercicio 2, el sistema se estructuró en dos entidades orientadas a objetos con funciones disjuntas:

3.3.1. Clase ContadorPalabras (Lógica de Dominio y Modelo)

La clase `ContadorPalabras` constituye el modelo puro del sistema, careciendo de vínculos con paquetes de interfaz gráfica. Su estado interno reúne tres atributos privados: el descriptor `archivo` de tipo `File`, la cadena `contenido` de tipo `String` para conservar el texto leído, y la variable entera `totalPalabras` para registrar el total cuantificado.

Su diseño algorítmico comprende los siguientes elementos:

- **Constructores:** Provee un constructor por defecto que inicializa el modelo en estado nulo o vacío (cero palabras y cadena vacía), y un constructor sobrecargado parametrizado que recibe un descriptor `File` e invoca de inmediato a la rutina de carga y validación.
- **Método `cargarArchivo(File archivoSeleccionado)`:** Ejecuta una secuencia exhaustiva de comprobaciones previas:
  1. Verifica que la referencia del archivo no sea nula, arrojando una excepción `IllegalArgumentException` si se transgrede la precondición.
  2. Normaliza el nombre del archivo a minúsculas mediante `toLowerCase()` y corrobora que termine estrictamente en el sufijo `.txt`, disparando `IllegalArgumentException` ante extensiones ajenas.
  3. Comprueba la presencia física del archivo en el sistema de almacenamiento mediante `exists()`, arrojando `FileNotFoundException` ante rutas no encontradas.
  4. Valida que la ruta corresponda a un archivo regular mediante `isFile()`, lanzando `IOException` ante fallos de tipificación.
  5. Realiza la lectura atómica del fichero completo mediante `Files.readString` bajo la codificación estándar `StandardCharsets.UTF_8`, asigna el contenido al atributo de instancia y despacha el cálculo de palabras.
- **Método `contarPalabras(String texto)`:** Implementa el algoritmo de tokenización léxica. Si la cadena recibida es nula o carece de caracteres útiles tras la aplicación de `trim()`, retorna cero de inmediato. Si contiene texto, remueve espacios en los bordes y segmenta el contenido aplicando la expresión regular `\s+` (*whitespace matching*), la cual colapsa secuencias continuas de espacios en blanco, tabulaciones y saltos de línea (`\n`, `\r`). La cantidad de elementos del arreglo resultante equivale de forma exacta al total de palabras computadas.

- **Métodos de acceso:** Expone métodos de lectura (`getArchivo`, `getContenido` y `getTotalPalabras`), asegurando el encapsulamiento del estado interno.

3.3.2. Clase VentanaContador (Interfaz Gráfica y Controlador)

La clase `VentanaContador` hereda de `JFrame`, funcionando como vista interactiva y despachador de eventos de usuario. Encapsula una referencia al modelo `ContadorPalabras` y gobierna los componentes de la interfaz: el botón `botonCargar` de tipo `JButton`, el área de texto `areaTexto` de tipo `JTextArea`, la etiqueta informativa `etiquetaTotal` de tipo `JLabel` y el contenedor con barras de desplazamiento `panelDesplazamiento` de tipo `JScrollPane`.

Las decisiones técnicas de diseño implementadas comprenden:

- **Disposición gráfica (*Layout*):** Se emplea un esquema `BorderLayout`. En la zona norte (`NORTH`) se aloja el botón de carga dentro de un panel superior; en la zona central (`CENTER`) se instala el `JScrollPane` que contiene al `JTextArea`; y en la zona sur (`SOUTH`) se presenta la etiqueta con el recuento de palabras.

- **Configuración del área de texto:** Para garantizar una adecuada presentación visual, el área de texto se configura en modo de solo lectura (`setEditable(false)`), se activa el corte de línea (`setLineWrap(true)`) con ajuste por palabras completas (`setWrapStyleWord(true)`) y se establece una tipografía monoespaciada uniforme.

- **Selector de archivos (`JFileChooser`):** La pulsación del botón activa un cuadro de diálogo interactivo provisto de un filtro `FileNameExtensionFilter` exclusivo para la extensión `.txt`, inhabilitando el filtro comodín de todos los archivos (`setAcceptAllFileFilterUsed(false)`) para restringir la selección a documentos válidos.

- **Manejo de excepciones y compatibilidad Headless:** Al confirmar la selección, el método `procesarArchivo(File)` delega la operación al modelo dentro de un bloque `try-catch` que intercepta `IOException` e `IllegalArgumentException`. Si el sistema opera en un entorno gráfico físico, proyecta una alerta con `JOptionPane.showMessageDialog`. Mediante la consulta a `GraphicsEnvironment.isHeadless()`, el programa permite además la ejecución automatizada de pruebas sin levantar componentes visuales incompatibles.

- **Método principal (`main`):** Incorpora soporte dual para ejecución en modo CLI (si se reciben argumentos con la ruta de un archivo) y modo gráfico interactivo gobernado en el hilo EDT mediante `SwingUtilities.invokeLater`.

3.4. Relaciones estructurales y diagramas UML

La arquitectura de clases se fundamenta en una relación de composición dirigida de cardinalidad uno a uno (1 a 1), en la cual `VentanaContador` posee e inicializa una instancia privada de `ContadorPalabras`. El modelo mantiene independencia absoluta respecto a los artefactos visuales de Swing.

En la figura 3.7 se detalla el diagrama UML global de clases que exhibe la arquitectura completa del software. Asimismo, en la figura 3.8 se presenta el diagrama de clase individual correspondiente a `ContadorPalabras`, mientras que en la figura 3.9 se ilustra la especificación interna de `VentanaContador`.

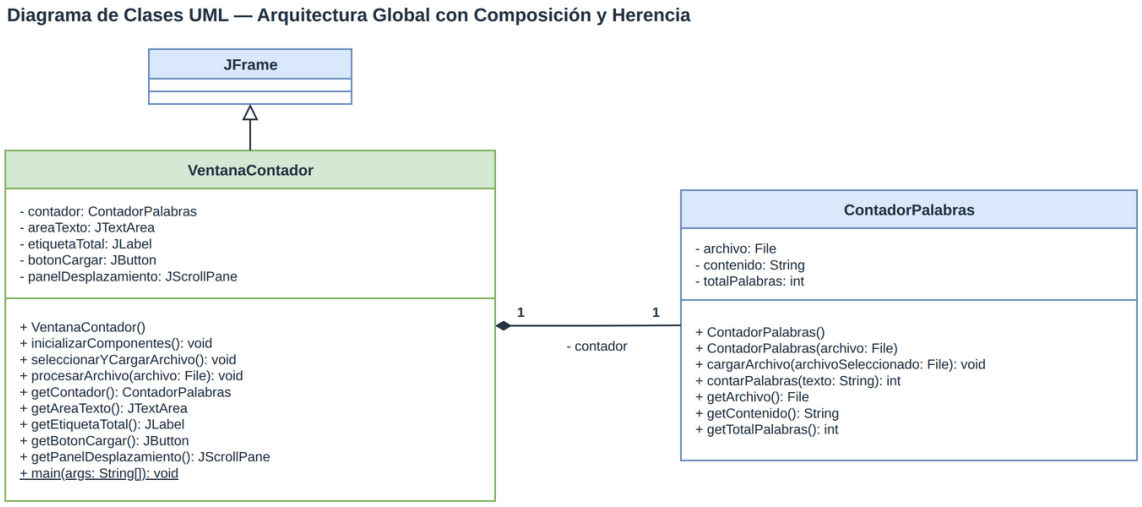


Figura 3.7: Diagrama UML global de clases

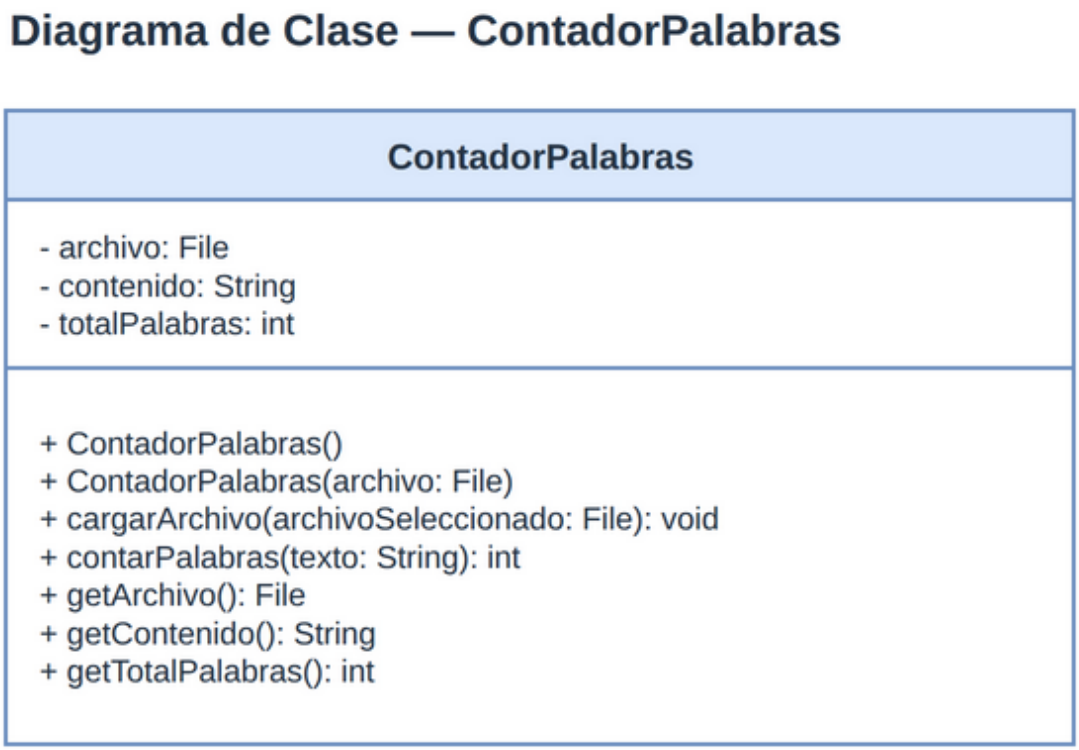


Figura 3.8: Diagrama UML de ContadorPalabras

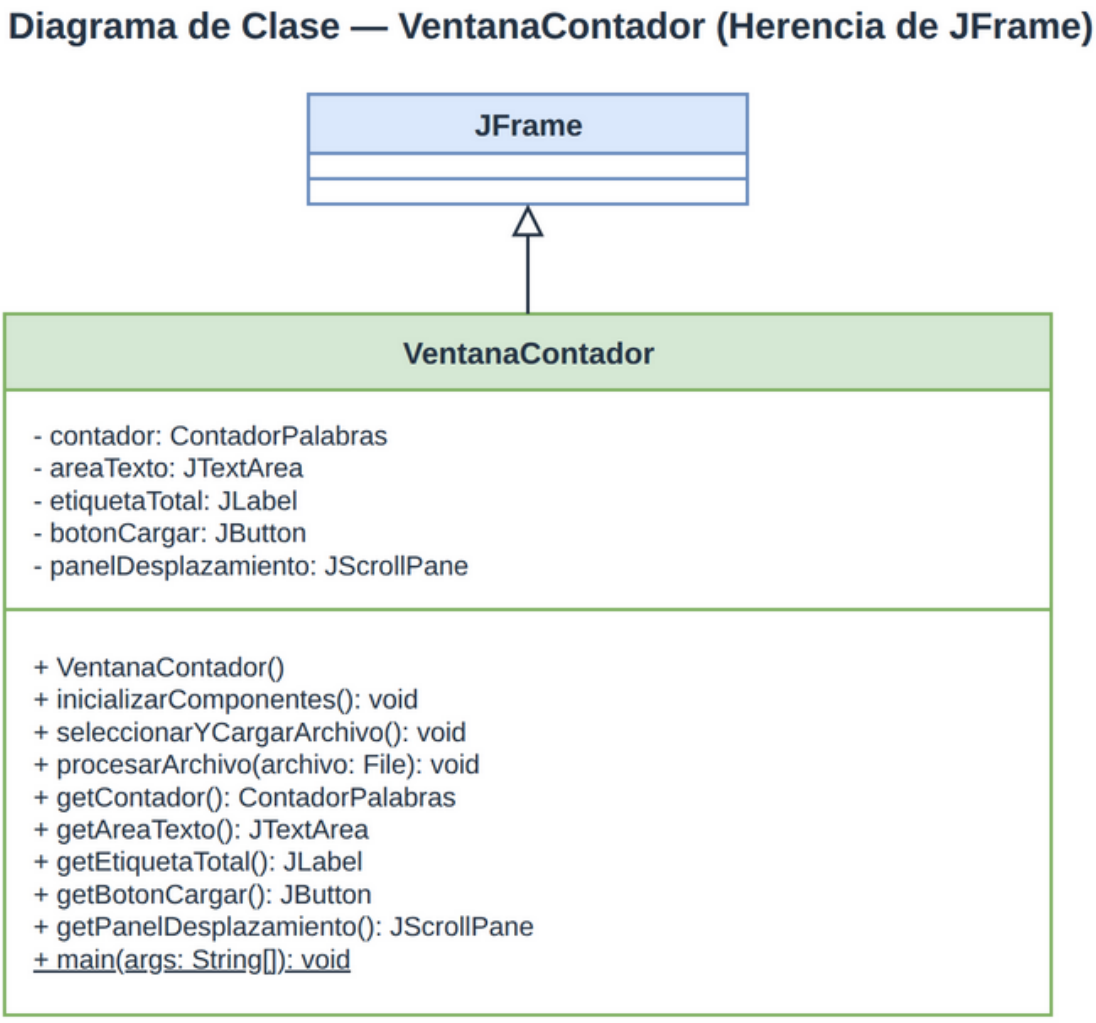


Figura 3.9: Diagrama UML de VentanaContador

3.5. Dinámica algorítmica y diagramas de comportamiento

El comportamiento dinámico del sistema se organiza a través de flujos estructurados de control y validación. En la figura 3.10 se representa el diagrama de flujo correspondiente al método `cargarArchivo` de la clase `ContadorPalabras`, detallando la secuencia de comprobaciones defensivas de nulidad, extensión, existencia física y lectura atómica bajo codificación UTF-8.

Por su parte, en la figura 3.11 se documenta el diagrama de flujo de la operación `procesarArchivo` en `VentanaContador`, evidenciando la delegación hacia el modelo, la actualización simultánea del área de texto y la etiqueta, y la captura controlada de excepciones.

Finalmente, en la figura 3.12 se formaliza el diagrama de secuencia que modela el intercambio temporal de mensajes entre el usuario, el despachador de eventos de `VentanaContador`, el selector interactivo `JFileChooser` y la clase `ContadorPalabras`.

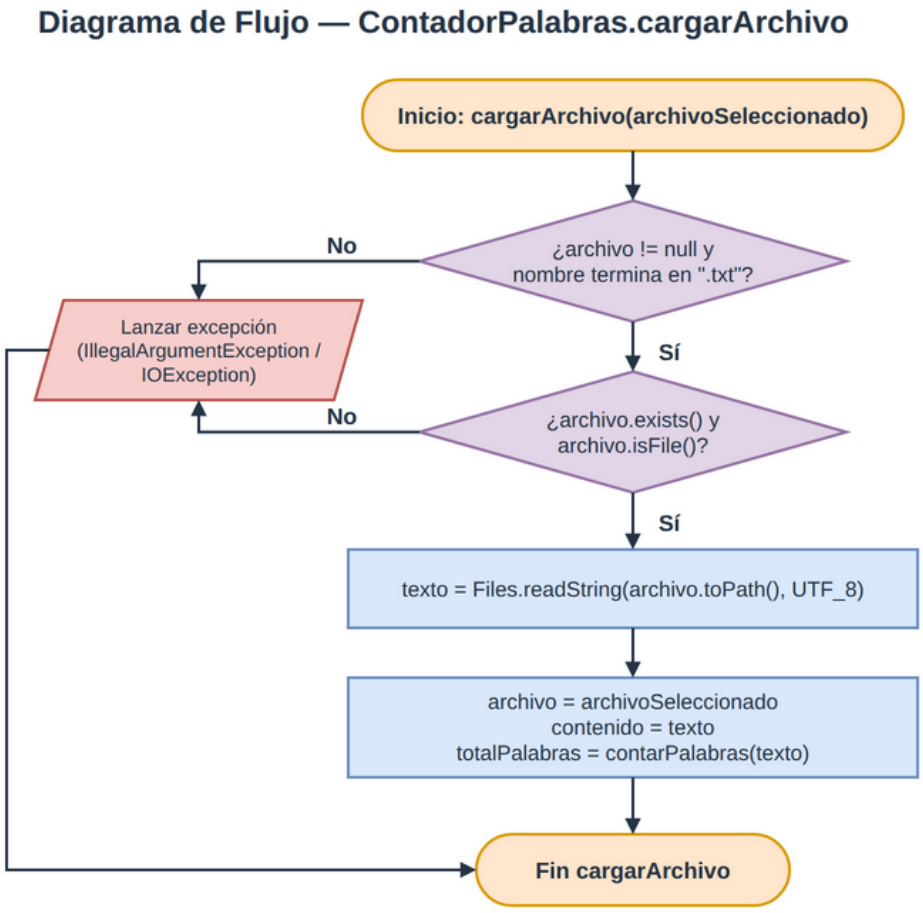


Figura 3.10: Diagrama de flujo de cargarArchivo

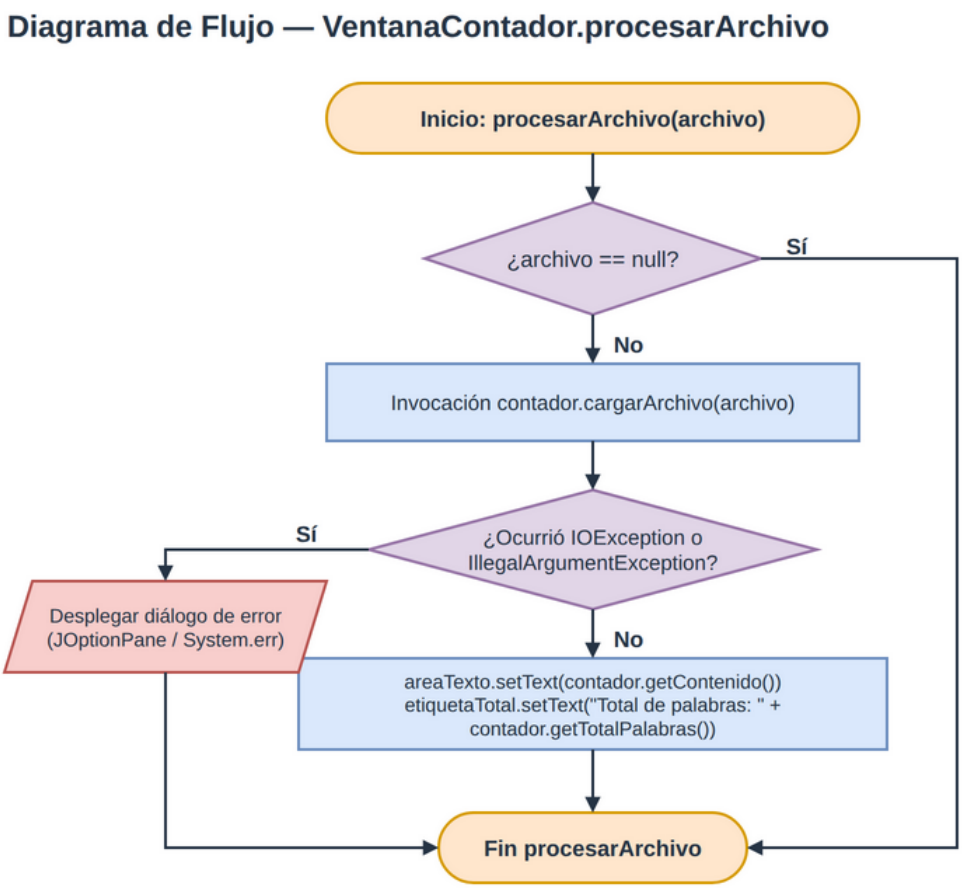


Figura 3.11: Diagrama de flujo de procesarArchivo

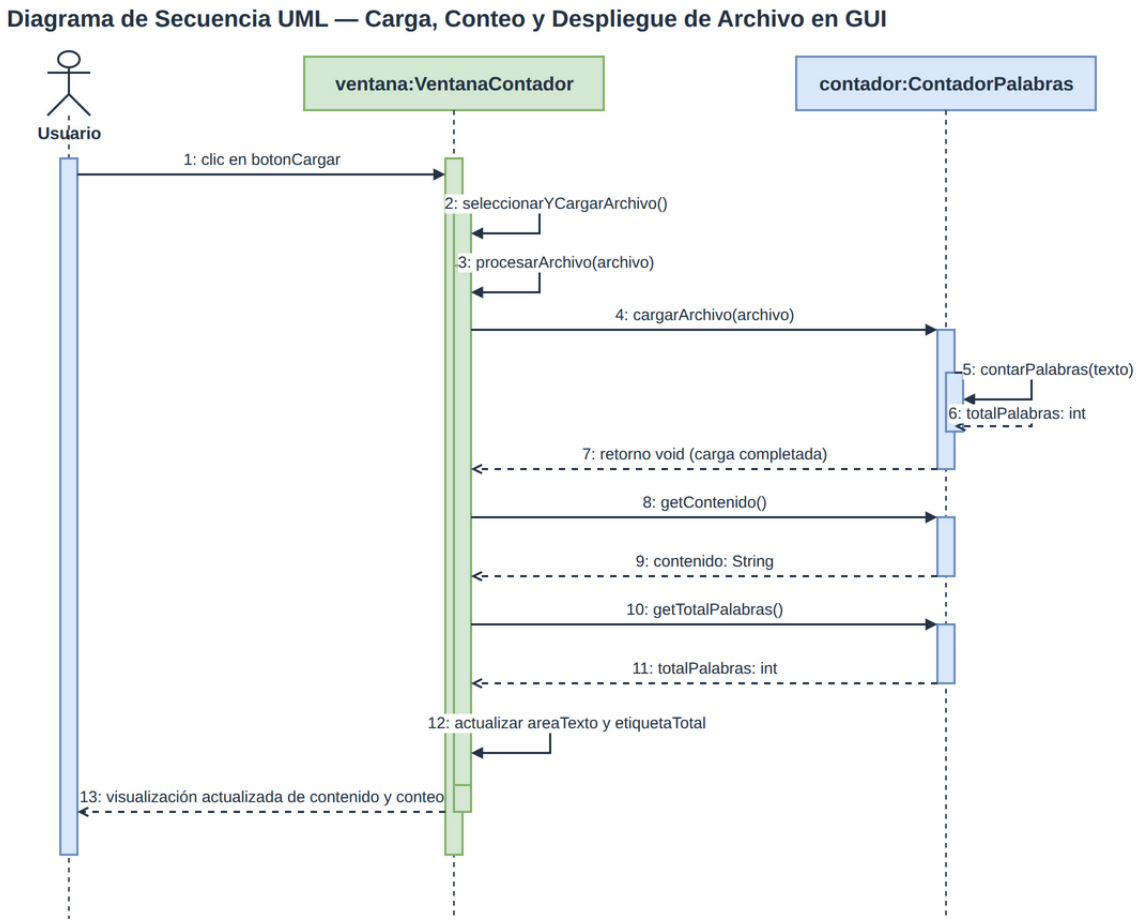


Figura 3.12: Diagrama de secuencia de lectura y actualización

4. Resultados

La comprobación experimental del sistema se realizó aplicando una batería rigurosa de pruebas sobre el paquete ejecutable generado, contrastando analíticamente los valores esperados frente a los observados en tiempo de ejecución. En la tabla 2 se sintetiza la matriz de casos analizados, detallando las entradas proporcionadas, el comportamiento observado y el veredicto funcional alcanzado.

Caso de Prueba	**Entrada Suministrada**	**Comportamiento Observado / Conteo**	**Estado**
Caso 1 (Válido) (`caso_valido.txt`)	Archivo estructurado en dos párrafos con puntuación y acentos ortográficos regulares.	Código de salida 0. Lectura atómica UTF-8 íntegra. Conteo exacto de 39 palabras delimitadas por espacios y saltos.	Aprobado
Caso 2 (Frontera) (`caso_frontera.txt`)	Archivo con espacios continuos múltiples, tabulaciones (`\t`), saltos repetidos, símbolos (¡!, ¿?%) y acentos (á, ó, ü, ñ).	Código de salida 0. Colapso determinista de espacios y tabuladores por expresión regular `\s+`. Conteo exacto de 19 palabras.	Aprobado

Tabla 2: Matriz de casos de prueba y verificación experimental del sistema.

4.1. Caso 1: Representativo (Texto estructurado en español)

El caso representativo analizó la lectura del archivo `pruebas/caso_valido.txt`, cuyo contenido aborda conceptos de programación orientada a objetos organizados en párrafos convencionales. El comportamiento esperado calculado analíticamente preveía el cómputo de 39 palabras bajo codificación UTF-8.

La ejecución observada confirmó con exactitud dicha predicción, reportando 39 palabras sin alteraciones y finalizando con código de salida 0. En la figura 4.1 se exhibe la captura de consola con la evidencia correspondiente. Asimismo, la ejecución interactiva en la interfaz gráfica corrobora la visualización íntegra del texto en el área desplazable `JTextArea` y el cómputo reactivo de la etiqueta `JLabel`, según se evidencia en la figura 4.2.

```
═══ CONTADOR DE PALABRAS GUI ═══
Archivo procesado: caso_valido.txt
Contenido:
----------------------------------------
La programacion orientada a objetos organiza el software en clases y objetos.
Cada objeto contiene atributos que representan su estado y metodos que definen su comportamiento.

Esta arquitectura facilita el mantenimiento, la modularidad y la reutilizacion del codigo fuente.
----------------------------------------
Total de palabras: 39
```

Figura 4.1: Ejecución del caso de prueba válido en terminal

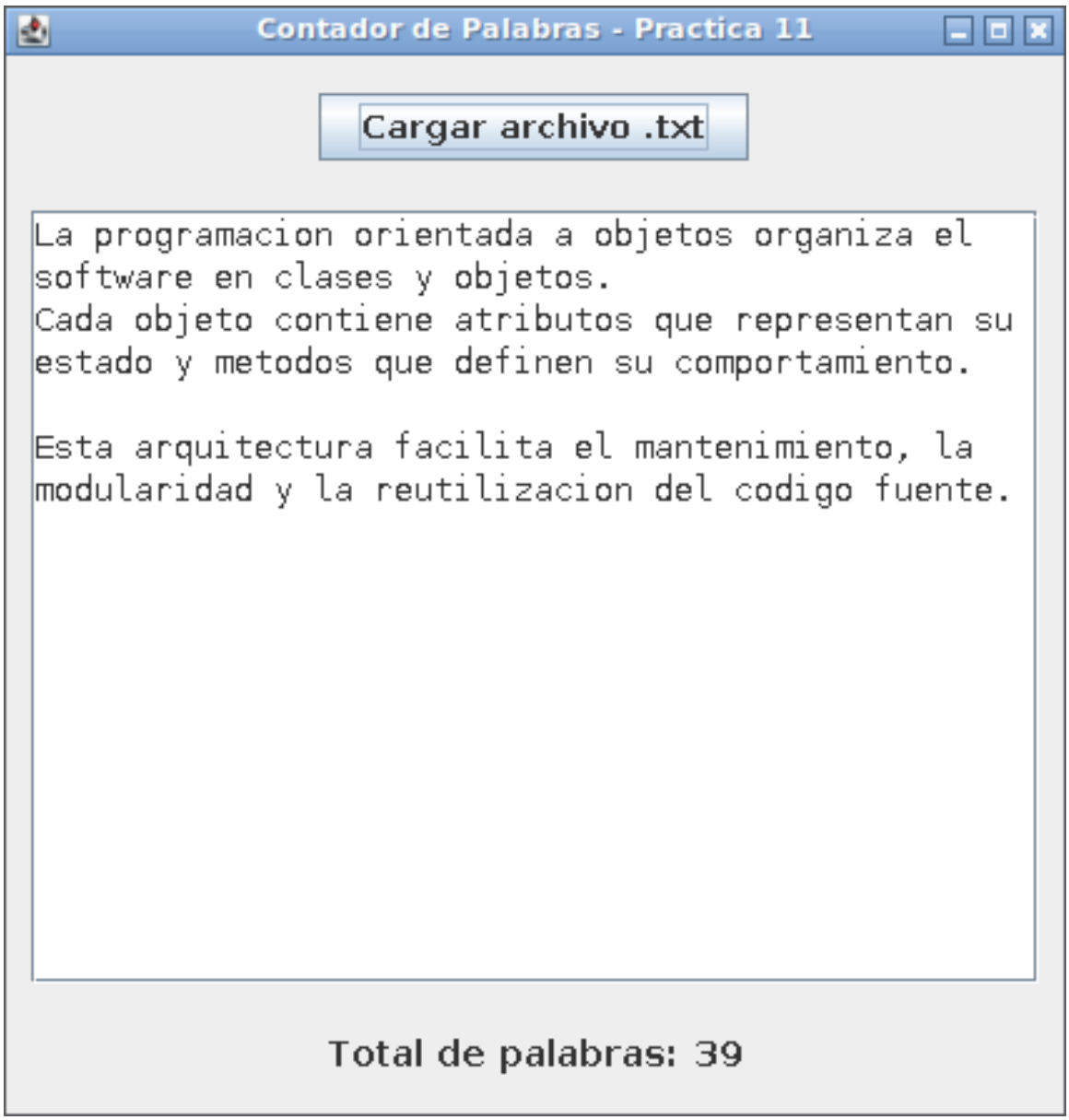


Figura 4.2: Visualización del caso representativo en la interfaz gráfica con 39 palabras

4.2. Caso 2: Frontera (Espaciados múltiples, tabuladores y caracteres especiales)

El caso frontera evaluó la robustez del algoritmo mediante el fichero `pruebas/caso_frontera.txt`, el cual contiene secuencias complejas compuestas por bloques continuos de espaciado en blanco, tabulaciones reiteradas (`\t`), líneas vacías intermedias, signos de admiración e interrogación invertidos (¡!, ¿?), porcentajes (100 %) y términos con vocales acentuadas, diéresis y la letra eñe (*águila, camión, pingüino, año*).

El cálculo analítico postulaba que la expresión regular `\s+` debía agrupar cualquier secuencia continua de espacios o tabuladores en un único separador sin originar tokens nulos, arrojando un total exacto de 19 palabras. La ejecución práctica corroboró plenamente dicho comportamiento, preservando la codificación diacrítica y cuantificando las 19 palabras de forma precisa, según se evidencia en la captura de consola de la figura 4.3. De manera análoga, en la modalidad gráfica la interfaz procesa de forma fidedigna el fichero frontera preservando la disposición de tabulaciones y caracteres especiales sin desbordar el contenedor, tal como se aprecia en la figura 4.4.

```
=== CONTADOR DE PALABRAS GUI ===
Archivo procesado: caso_frontera.txt
Contenido:
----------------------------------------
   ¡Hola,   mundo!              Este            es   un    caso   frontera.

Tabuladores     y    multiples    espacios.

¿Funciona?   Si,   100%   garantizado:   águila,   camión,   pingüino,   año.
----------------------------------------
Total de palabras: 19
```

Figura 4.3: Ejecución del caso de prueba frontera en terminal

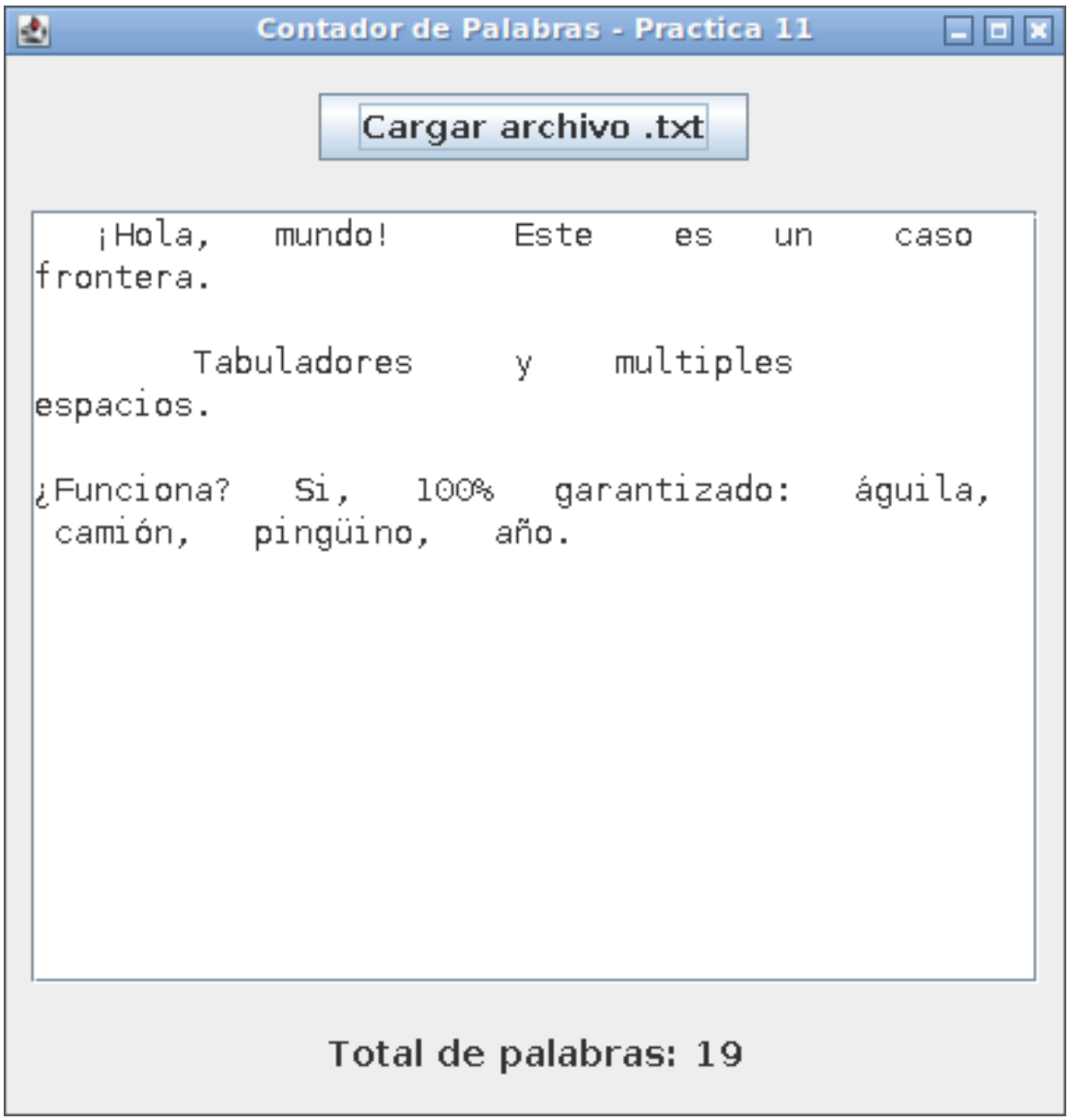


Figura 4.4: Visualización del caso frontera en la interfaz gráfica con 19 palabras

4.3. Verificación del estado inicial de la interfaz gráfica de usuario

Previo a la invocación del diálogo selector de ficheros `JFileChooser`, la aplicación inicializa la ventana `VentanaContador` en un estado neutro y consistente. El área de texto permanece limpia sin contenidos residuales, el botón de carga se encuentra habilitado para la interacción del usuario y la etiqueta inferior fija el marcador en cero palabras. En la figura 4.5 se presenta la captura de la ventana en dicho estado basal.

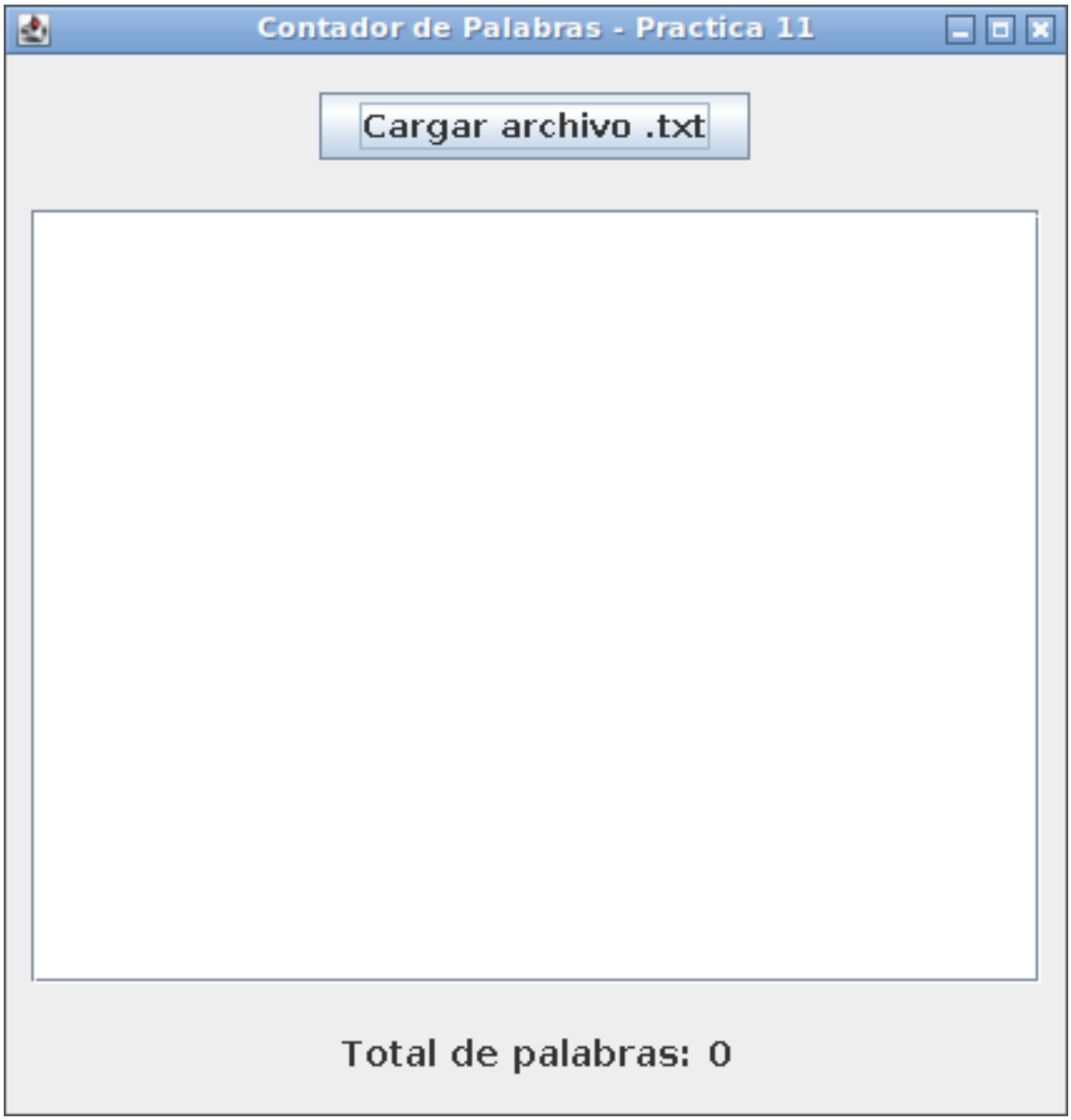


Figura 4.5: Estado inicial de la interfaz gráfica VentanaContador

4.4. Verificación de empaquetado en archivo JAR autónomo

La fase de verificación experimental culminó con la validación del binario empaquetado `lib/practica11.jar`. La construcción del empaquetado autónomo se realizó mediante la herramienta nativa `jar` del OpenJDK, configurando el descriptor de manifiesto `META-INF/MANIFEST.MF` con la directiva obligatoria:

```
Main-Class: VentanaContador
```

Dicha directiva faculta la invocación directa del software a través de la máquina virtual de Java mediante la instrucción `java -jar`, prescindiendo de la especificación manual del cargador de clases en el parámetro de *classpath*.

Para corroborar la portabilidad y determinismo del componente binario, se ejecutó en terminal el comando de validación:

```
java -jar lib/practica11.jar pruebas/caso_valido.txt
```

comprobando que el paquete autónomo resuelve con éxito tanto el procesamiento reproducible por consola (emitiendo en la salida estándar el reporte formateado con 39 palabras contabilizadas y finalizando con código de salida 0) como la inicialización reactiva de la interfaz visual `VentanaContador` en entornos provistos de servidor gráfico.

Asimismo, se verificó la integridad del archivo JAR mediante la inspección exhaustiva de su tabla de contenidos (`jar -tf lib/practica11.jar`), constatando la inclusión exclusiva de las clases finales compiladas `ContadorPalabras.class` y `VentanaContador.class`, junto con el árbol de metadatos del manifiesto. De este modo, se garantiza un artefacto autónomo, ligero y desprovisto de dependencias externas no estándar.

5. Conclusiones

La realización de la presente práctica ratifica que la separación rigurosa entre la lógica de dominio y la capa de presentación constituye un principio indispensable para la ingeniería de software orientada a objetos. Al encapsular la validación de archivos, la lectura física y el procesamiento léxico en una clase de modelo independiente como `ContadorPalabras`, se asegura la portabilidad y reusabilidad del componente algorítmico, facultando su comprobación sistemática en entornos de terminal desasistidos y posibilitando la evolución tecnológica de la interfaz gráfica sin alterar el núcleo operativo.

Por otra parte, el análisis comparativo entre las bibliotecas gráficas en Java evidencia la transición desde esquemas tradicionales basados en componentes ligeros renderizados por software en la CPU bajo el Event Dispatch Thread, como en Swing, hacia plataformas contemporáneas como JavaFX fundamentadas en Grafos de Escena acelerados por hardware en la GPU y enlaces reactivos de propiedades (*Property Binding*). Comprender la necesidad de envolver componentes como `JTextArea` dentro de decoradores como `JScrollPane` permite interiorizar los principios de diseño por composición frente a la herencia en la construcción de interfaces robustas.

Finalmente, el empleo de flujos contemporáneos mediante Java NIO.2 y el método `Files.readString` pone de relieve la trascendencia de gobernar explícitamente la codificación de caracteres en el estándar UTF-8. Dicha disciplina, complementada con el colapso determinista de espacios y tabuladores arbitrarios a través de expresiones regulares, dota al software de resiliencia frente a contingencias del sistema operativo o entradas complejas en la frontera del dominio de cómputo.

Referencias

[1] Gosling, J., Joy, B., Steele, G., Bracha, G., Buckley, A., Smith, D., & Bierman, G. (2023). *The Java Language Specification, Java SE 21 Edition*. Oracle America, Inc.

[2] Oracle Corporation. (2022). *Using Swing Components: Text Component Features and Concurrency in Swing*. The Java Tutorials, Oracle America, Inc.

[3] OpenJFX Community. (2023). *JavaFX Controls, Architecture and Scene Graph Documentation*. Eclipse Foundation & Oracle Corporation.

[4] Oracle Corporation. (2023). *Java Platform, Standard Edition Version 21 API Specification: Package java.nio.file*. Oracle America, Inc.

[5] Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley Professional.